

LLD Practice Platform

Design Note — MVP, User Flow, Classes & Trade-offs

1. MVP Overview

The MVP is a single practice loop: a learner picks from three seeded LLD problems (Parking Lot, Elevator, Vending Machine), submits a free-text design description, and receives structured, rubric-based feedback generated by an LLM. The learner can review past attempts to see whether the same weaknesses keep showing up.

Scope is deliberately narrow. In: three problems, text-only submissions, one AI evaluator behind a swappable interface, and attempt history. Out: authentication, code execution or compilation, diagram/image submissions, and multi-evaluator consensus. Everything excluded was cut because it would add build risk without adding evidence of design quality — the thing this assignment is actually graded on.

2. User Flow

The flow below is intentionally linear with one branch. Evaluation never blocks the submission request — the Attempt is persisted as **SUBMITTED** immediately, so a slow or failed LLM call can never lose a learner's work. A failed evaluation loops the learner straight back to resubmitting, rather than dead-ending the attempt.

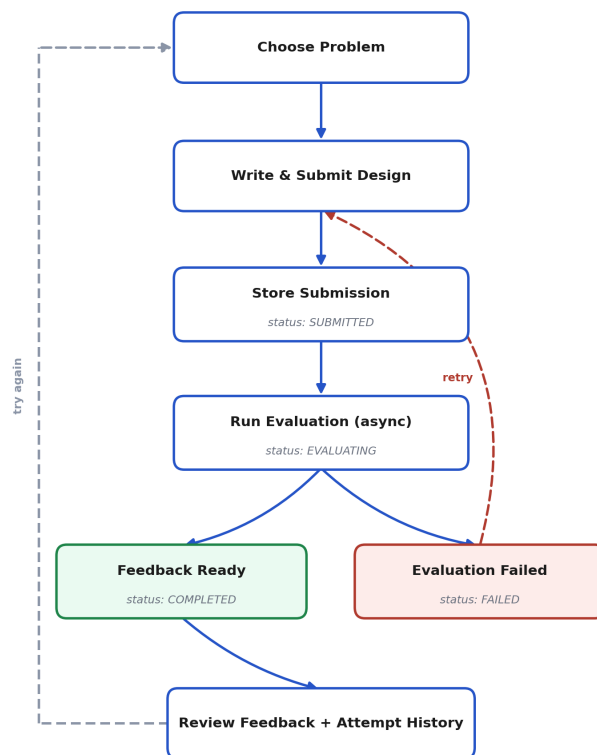


Figure 1 — Practice loop and attempt state transitions

3. Core Domain Classes

The domain model keeps **Attempt** as the central object tying a Problem, a Submission, and an Evaluation together through one status lifecycle. Evaluation logic sits behind an **Evaluator** interface, with **LlmEvaluator** as the only

implementation today — this is the one abstraction added purely for future change, since a rule-based or human evaluator can be dropped in later without touching the Attempt flow.

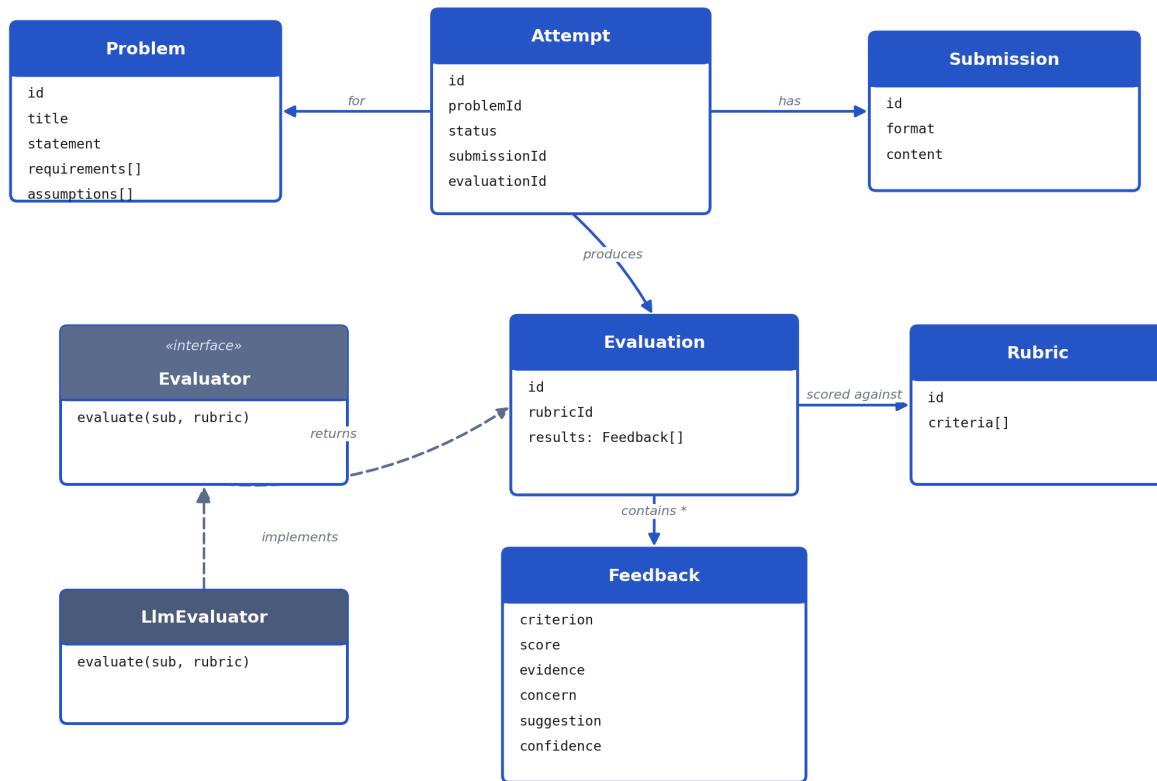


Figure 2 — Domain model and class relationships

Class	Responsibility
Problem	Immutable seed content a learner attempts against; owns nothing else.
Attempt	Ties one Problem, Submission, and Evaluation together; owns the status lifecycle.
Submission	Learner-provided content; carries a format field so new formats don't require a new entity.
Evaluator (interface)	Contract for turning a Submission + Rubric into an Evaluation; decouples the flow from any one evaluation strategy.
Evaluation / Feedback	Structured, per-criterion results (score, evidence, concern, suggestion) — never a single opaque score.

4. Evaluation Approach

Deterministic checks (non-empty submission, valid state transitions, no duplicate processing) run in plain code with no AI involved. Judgment-heavy checks — responsibility quality, trade-offs, extensibility, testability, quality of explanation — go to the LLM, which is always given the fixed rubric and required to return structured JSON (criterion → score → evidence → concern → suggestion → confidence). The model is never asked an open-ended “is this good?” question.

5. Key Trade-offs

Decision	What We Chose	Trade-off Accepted
Submission format	Text-only design description	Faster to build and evaluate reliably; can't verify code compiles, but text is enough to assess responsibilities and reasoning for an MVP.
Evaluation timing	Async — store first, evaluate after	Never blocks the request or loses a submission on evaluator failure; adds status polling on the frontend.
AI prompting	Fixed rubric + required structured JSON	Consistent, comparable feedback across attempts; costs upfront rubric design effort.
Evaluator design	Interface with one implementation	Adds one layer of indirection today, but a second evaluator can be added without rewriting the practice flow.
Problem set	3 fixed, well-known problems	Limits variety, but keeps the MVP scoped to what's provable in two days.